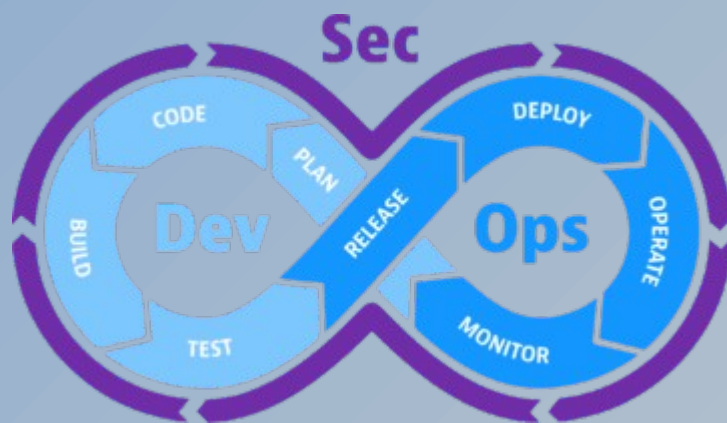


Laboratorio DevSecOps



Pedro Egea Ortega

Índice

Actividad 1.....	3
1. Configuración inicial del entorno.....	3
2. Despliegue de los servicios con Docker.....	3
3. Acceso a DVWA y reseteo de la base de datos.....	4
4. Instalación de Docker en el contenedor de Jenkins.....	5
5. Puesta en marcha de Jenkins.....	6
6. Puesta en marcha de SonarQube y DefectDojo.....	7
Actividad 2.....	8
1. Configuración del token de DefectDojo en Jenkins.....	8
2. Añadir el Product Type en DefectDojo.....	9
3. Añadir el Producto en DefectDojo.....	9
4. Añadir el Engagement en DefectDojo.....	11
5. Creación del Pipeline en Jenkins.....	12
6. Verificación de resultados en DefectDojo.....	14
7. Añadir un Engagement Interactive para escaneos manuales.....	15
8. Añadir un Test con Nikto.....	16
9. Verificación de resultados en DefectDojo.....	17
Actividad 3.....	18
1. Generación del token en SonarQube.....	18
2. Configuración del token de SonarQube en Jenkins.....	19
3. Configuración del servidor SonarQube en Jenkins.....	20
4. Configuración de SonarScanner en Jenkins.....	21
5. Configuración del servidor SonarQube en DefectDojo.....	22
6. Creación del Engagement en DefectDojo.....	24
7. Creación del proyecto en SonarQube.....	25
8. Configuración de la API Scan Configuration en el Engagement.....	26
9. Creación del Pipeline en Jenkins.....	27
10. Verificación de resultados.....	28
Actividad 4.....	29
1. Consideraciones previas.....	29
2. Creación del Engagement en DefectDojo.....	29
3. Creación del Pipeline en Jenkins.....	30
4. Verificación de resultados en DefectDojo.....	31
Actividad 5.....	32
1. Consideraciones previas.....	32
2. Creación del Engagement en DefectDojo.....	32
3. Creación del Pipeline en Jenkins.....	33
4. Verificación de resultados en DefectDojo.....	34

Actividad 1

Realiza la puesta en marcha del Laboratorio DevSecOps Marina Mango como se expone en los apuntes.

1. Configuración inicial del entorno

El primer paso consiste en configurar correctamente la dirección IP del entorno de trabajo. Para ello, se debe modificar el archivo de variables de entorno ubicado en la siguiente ruta:

/home/ciber/devsecops-lab/.env

```
GNU nano 7.2 devsecops-lab/.env
MY_EXTERNAL_IP=192.168.100.116
```

Dentro de este archivo, se reemplaza la dirección IP existente por la dirección IP correspondiente a la máquina virtual que se está utilizando. Este cambio es necesario para asegurar la correcta comunicación entre los servicios desplegados en el laboratorio.

2. Despliegue de los servicios con Docker

Una vez configurada la dirección IP, se procede a levantar el entorno utilizando Docker Compose. Para ello, se ejecuta el siguiente comando desde el directorio del proyecto:

```
$ docker compose up -d --force-recreate
```

Tardará unos 10 minutos en acabar.

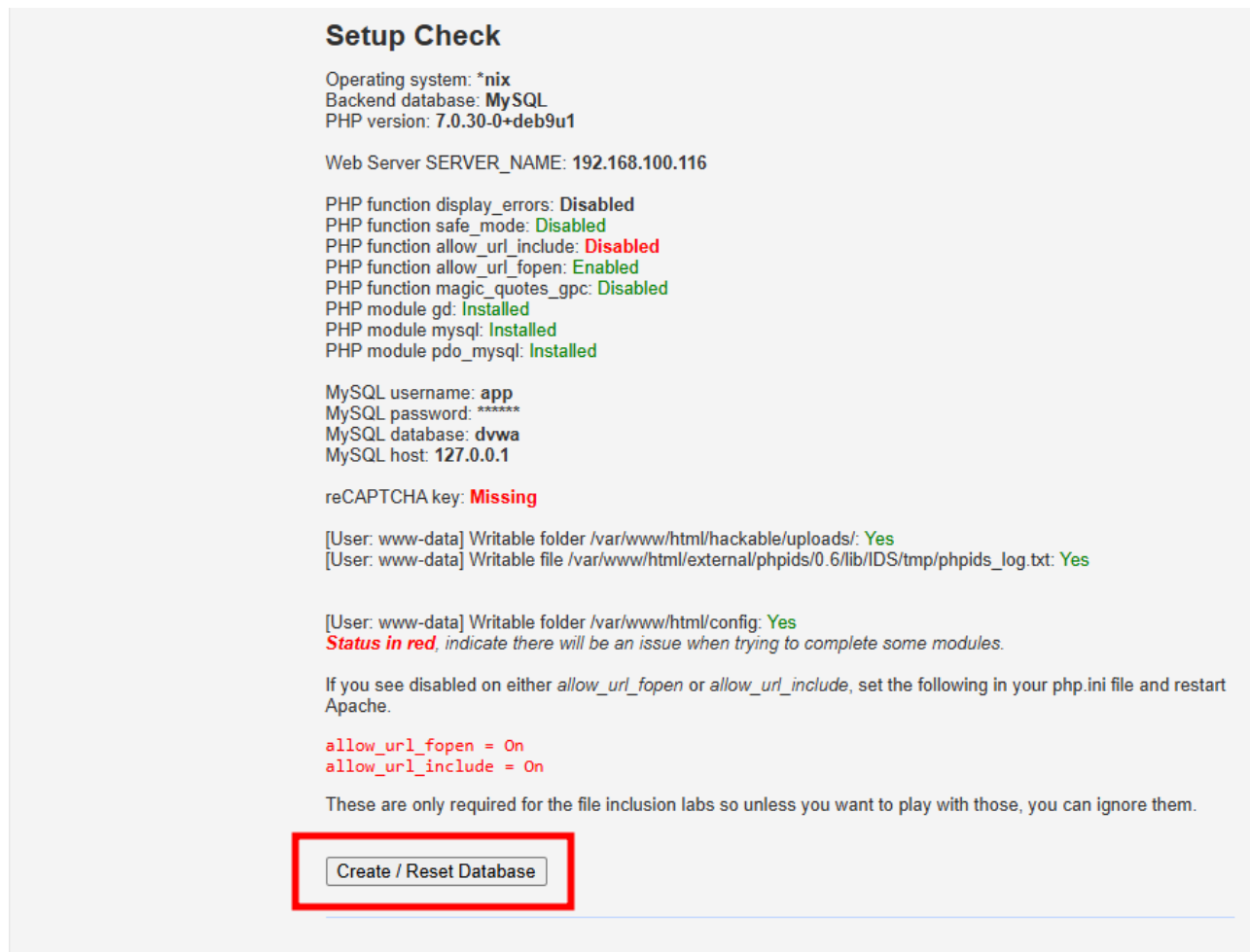
```
ciber@ciber:~/devsecops-lab$ docker compose up -d --force-recreate
[+] up 12/12
✓ Container dvwa           started
✓ Container trivy          started
✓ Container sonarqube      started
✓ Container dojo_redis     started
✓ Container jenkins        started
✓ Container dojo_postgres  healthy
✓ Container zap            started
✓ Container semgrep        started
✓ Container dojo_initializer started
✓ Container dojo_uwsgi      started
✓ Container dojo_celery     started
✓ Container dojo_nginx     started
ciber@ciber:~/devsecops-lab$
```

3. Acceso a DVWA y reseteo de la base de datos

Una vez que todos los servicios están completamente inicializados, se procede a acceder a la aplicación DVWA (Damn Vulnerable Web Application) desde el navegador web.

Para ello, se introduce en el navegador la dirección correspondiente al entorno (utilizando la IP configurada previamente):

<http://192.168.100.116:8081>



Setup Check

Operating system: *nix
Backend database: MySQL
PHP version: 7.0.30-0+deb9u1

Web Server SERVER_NAME: 192.168.100.116

PHP function display_errors: Disabled
PHP function safe_mode: Disabled
PHP function allow_url_include: Disabled
PHP function allow_url_fopen: Enabled
PHP function magic_quotes_gpc: Disabled
PHP module gd: Installed
PHP module mysql: Installed
PHP module pdo_mysql: Installed

MySQL username: app
MySQL password: *****
MySQL database: dvwa
MySQL host: 127.0.0.1

reCAPTCHA key: Missing

[User: www-data] Writable folder /var/www/html/hackable/uploads/: Yes
[User: www-data] Writable file /var/www/html/external/phpids/0.6/lib/IDS/tmp/phpids_log.txt: Yes

[User: www-data] Writable folder /var/www/html/config: Yes
Status in red, indicate there will be an issue when trying to complete some modules.

If you see disabled on either `allow_url_fopen` or `allow_url_include`, set the following in your php.ini file and restart Apache.

`allow_url_fopen = On`
`allow_url_include = On`

These are only required for the file inclusion labs so unless you want to play with those, you can ignore them.

Create / Reset Database

4. Instalación de Docker en el contenedor de Jenkins

Para permitir que Jenkins pueda interactuar con los demás contenedores del laboratorio, es necesario instalar Docker dentro de su contenedor.

Este paso es fundamental, ya que permitirá a Jenkins ejecutar comandos Docker y gestionar otros servicios durante las tareas de automatización que se configurarán más adelante.

El proceso consiste en acceder al contenedor de Jenkins y realizar la instalación de Docker en su interior. Una vez instalado, Jenkins podrá comunicarse con el resto de contenedores del entorno.

```
$ docker exec -u 0 -it jenkins bash
```

```
$ apt-get update && apt-get install -y
```

```
$ docker.io
```

```
$ apt install git -y
```

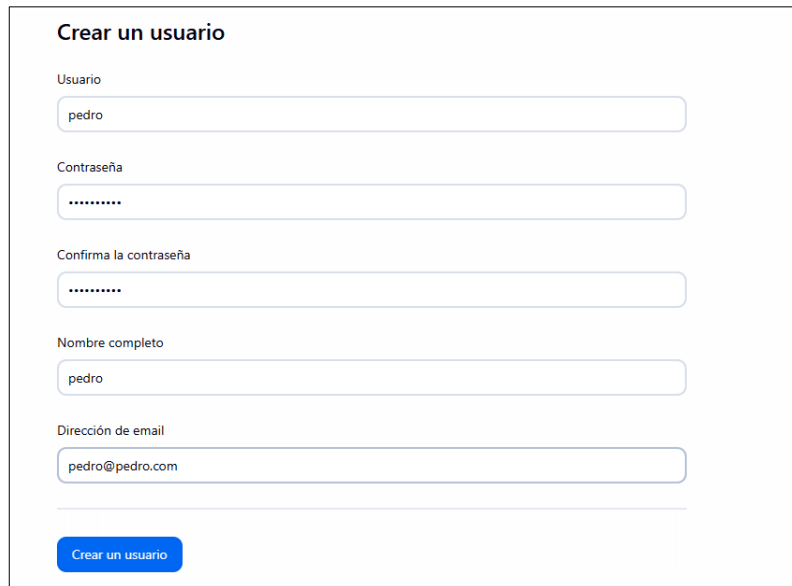
```
$ exit
```

```
ciber@ciber:~/devsecops-lab$ docker exec -u 0 -it jenkins bash
root@13012b411897:/# apt-get update && apt-get install -y
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [127 kB]
Fetched 10.0 MB in 1s (9338 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
0 upgraded, 0 newly installed, 0 to remove and 16 not upgraded.
root@13012b411897:/# docker.io
bash: docker.io: command not found
root@13012b411897:/# apt install git -y
git is already the newest version (1:2.47.3-0+deb13u1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 16
root@13012b411897:/# exit
exit
ciber@ciber:~/devsecops-lab$ |
```

5. Puesta en marcha de Jenkins

Una vez completada la instalación de Docker en el contenedor de Jenkins, se procede a poner en funcionamiento el servicio para comenzar a trabajar con él.

Primero creamos un nuevo usuario

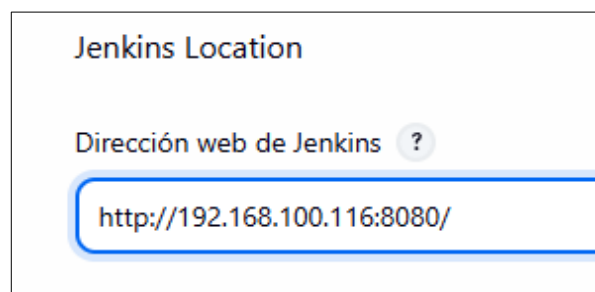


The screenshot shows the 'Crear un usuario' (Create a user) form in Jenkins. It contains the following fields and values:

- Usuario:** pedro
- Contraseña:** (masked with dots)
- Confirma la contraseña:** (masked with dots)
- Nombre completo:** pedro
- Dirección de email:** pedro@pedro.com

A blue button labeled 'Crear un usuario' is at the bottom.

Después cambiamos la ip en Administrar Jenkins > System en Jenkins Location y SonarQube Servers



The screenshot shows the 'Jenkins Location' configuration page. The 'Dirección web de Jenkins' (Jenkins web address) field is highlighted with a blue border and contains the value: `http://192.168.100.116:8080/`. A question mark icon is visible next to the field label.



The screenshot shows the configuration page for SonarQube. It contains the following fields and values:

- Name:** SonarQube
- URL del servidor:**
Por defecto es `http://localhost:9000`
`http://192.168.100.116:9000`

6. Puesta en marcha de SonarQube y DefectDojo

Iniciamos sesión en SonarQube con admin:admin y cambiamos la contraseña por [Ciber@2026](#)

Una vez desplegado el entorno, se realiza la configuración inicial de DefectDojo, comenzando por la obtención de credenciales y la creación del usuario administrador.

```
$ docker exec -it dojo_uwsgi python manage.py createsuperuser
```

Yo he creado el usuario pedro con la contraseña [Ciber@2026](#)

```
ciber@ciber:~/devsecops-lab$ docker exec -it dojo_uwsgi python manage.py createsuperuser
[24/Apr/2026 18:20:10] INFO [dojo.models:4572] enabling audit logging
Username (leave blank to use 'defectdojo'): pedro
Email address:
Password:
Password (again):
[24/Apr/2026 18:20:42] INFO [dojo.utils:1753] creating default set of notifications for: pedro
Superuser created successfully.
ciber@ciber:~/devsecops-lab$
```

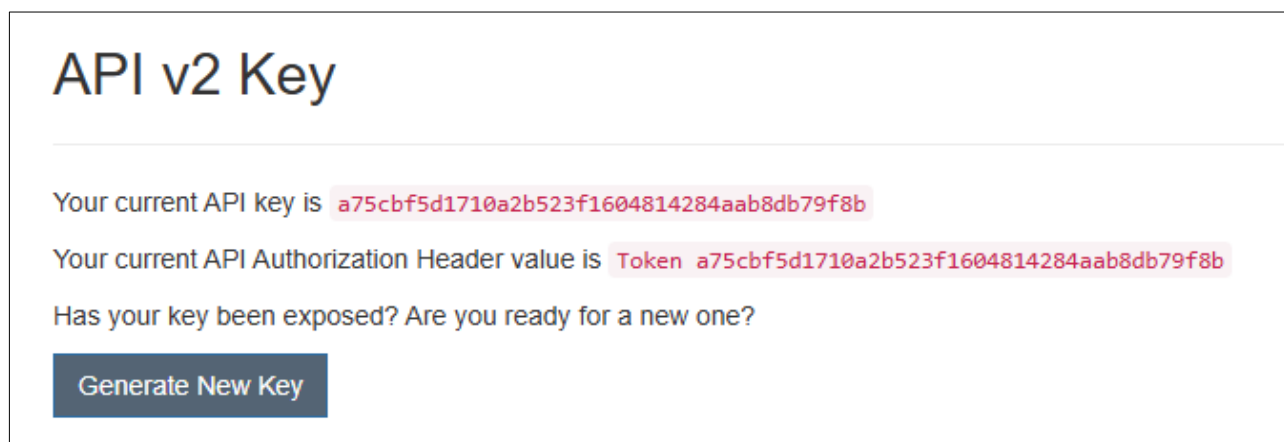
Actividad 2

DAST con ZAP

1. Configuración del token de DefectDojo en Jenkins

Vamos a comenzar realizando un **análisis dinámico (DAST)** de DVWA con ZAP, automatizándolo con Jenkins y centralizando la información en DefectDojo. Para ello, vamos a utilizar la aplicación DVWA que se encuentra en el contenedor y que es funcional: **http://[IP]:8081**

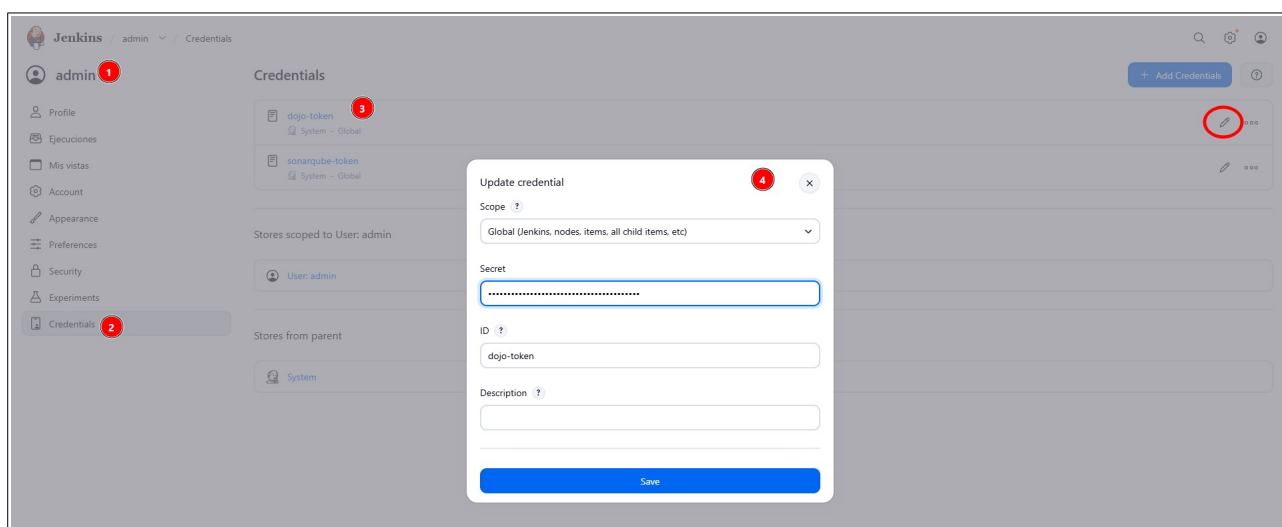
El primer paso consiste en obtener el token de autenticación de DefectDojo para poder integrarlo con Jenkins. Para ello, se accede a DefectDojo con el usuario administrador y en el menú superior derecho se selecciona la opción API v2 Key, donde se muestra el token de autenticación.



Your current API key is **a75cbf5d1710a2b523f1604814284aab8db79f8b**

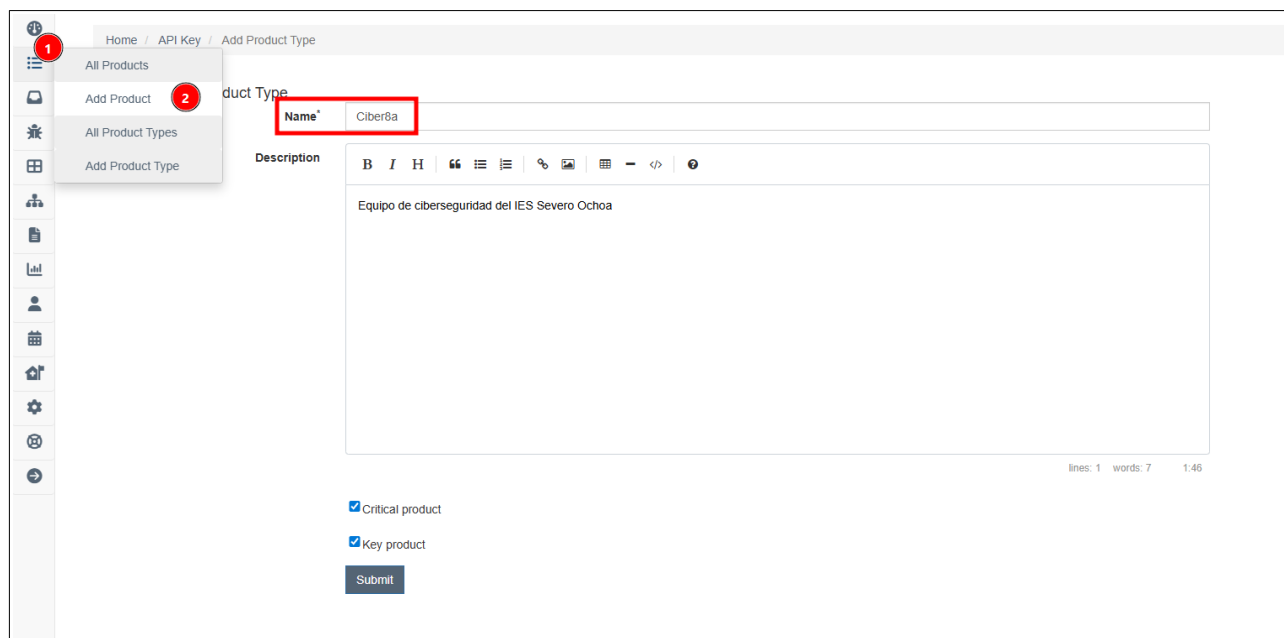
Your current API Authorization Header value is **Token a75cbf5d1710a2b523f1604814284aab8db79f8b**

Una vez copiado el token, se accede a Jenkins y se navega hasta Administrar Jenkins > Credenciales > Sistema > Global > Add Credentials. Se selecciona el tipo Secret Text, se pega el token en el campo Secret y se asigna el identificador dojo-token en el campo ID.



2. Añadir el Product Type en DefectDojo

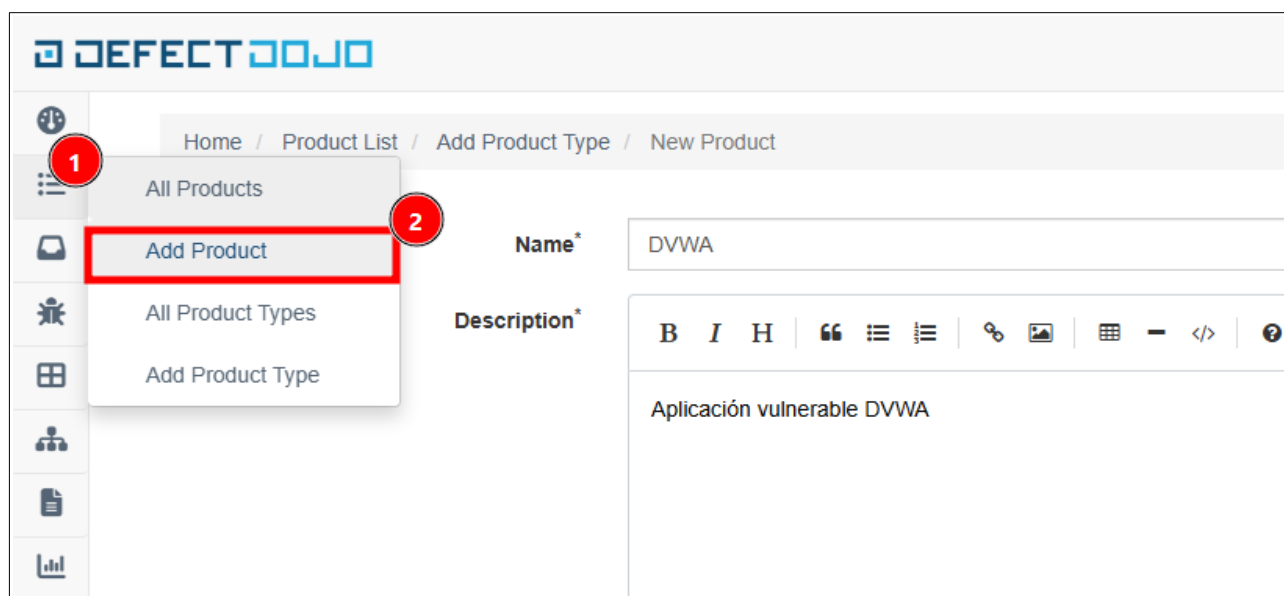
A continuación se configura la estructura de DefectDojo para centralizar los resultados. El primer elemento a crear es el Product Type, que representa la categoría del producto a analizar. Para ello se accede al menú lateral y se selecciona Add Product Type, asignando el nombre Ciber8a y como descripción Equipo de ciberseguridad del IES Severo Ochoa.



The screenshot shows the 'Add Product Type' form in DefectDojo. The breadcrumb trail is 'Home / API Key / Add Product Type'. The left sidebar has a red circle '1' next to the menu icon. The 'Add Product Type' menu item is highlighted with a red circle '2'. The 'Name*' field contains 'Ciber8a'. The 'Description' field contains 'Equipo de ciberseguridad del IES Severo Ochoa'. The 'Critical product' and 'Key product' checkboxes are checked. A 'Submit' button is at the bottom.

3. Añadir el Producto en DefectDojo

Una vez creado el Product Type, se procede a crear el Producto que representa la aplicación que se va a analizar. Se selecciona Add Product y se rellena el formulario con los siguientes datos: nombre DVWA, descripción Aplicación vulnerable DVWA, Product Type Ciber8a, Business Criticality High, Platform Web, Lifecycle Production y Origin Open Source.

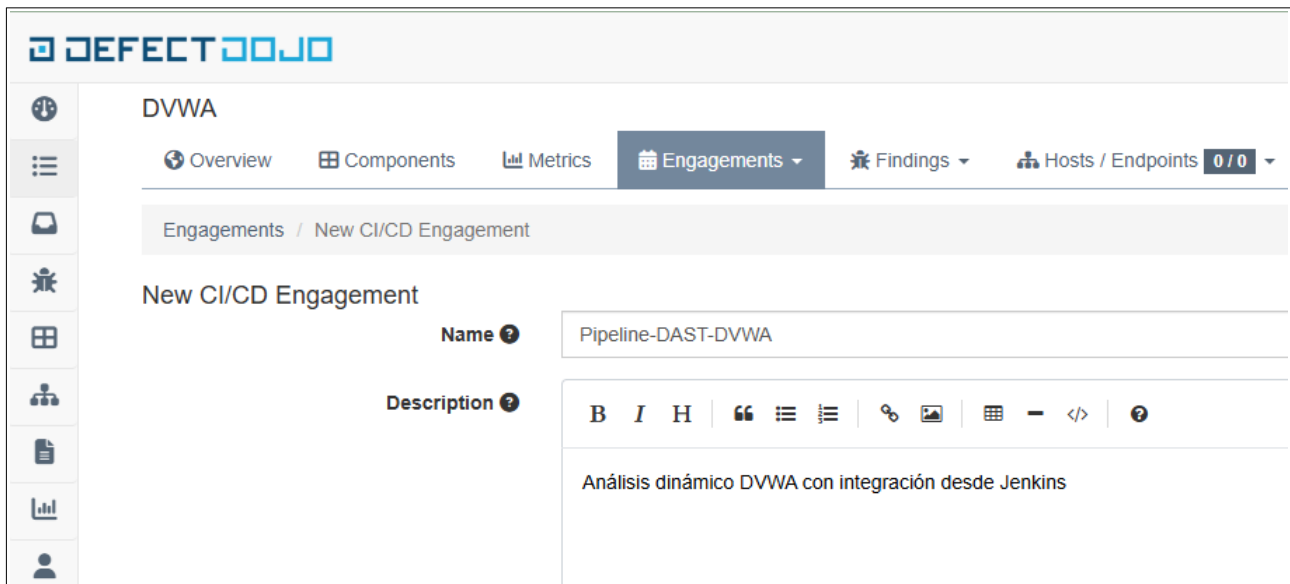
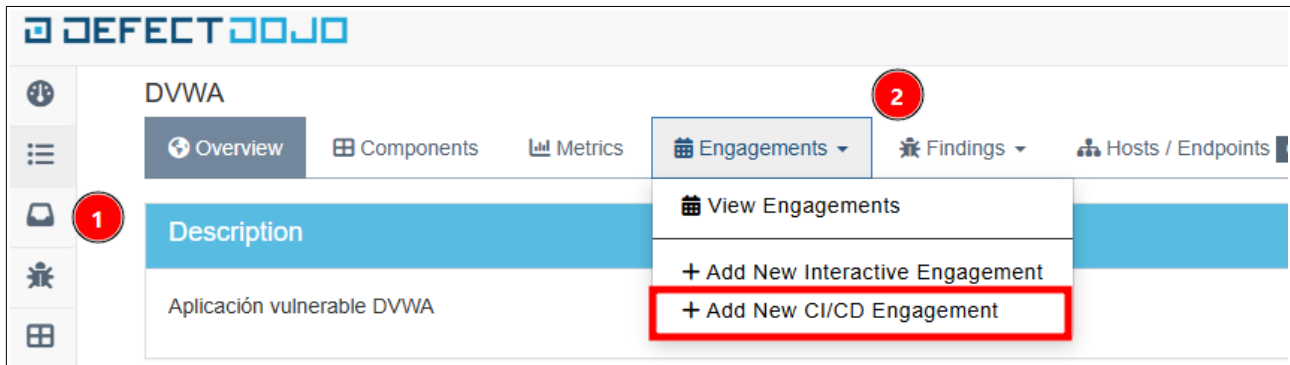


The screenshot shows the 'Add Product' form in DefectDojo. The breadcrumb trail is 'Home / Product List / Add Product Type / New Product'. The left sidebar has a red circle '1' next to the menu icon. The 'Add Product' menu item is highlighted with a red circle '2'. The 'Name*' field contains 'DVWA'. The 'Description*' field contains 'Aplicación vulnerable DVWA'.

Tags ?	Enter some tags (comma separated, use enter to select / create a new tag)
Product manager	(pedro)
Technical contact	(pedro)
Team manager	(pedro)
Product Type *	Ciber8a
SLA Configuration *	Default
Regulations	Nothing selected
Business criticality	High
Platform	Web
Lifecycle	Production
Origin	Open Source
User records ?	
Revenue ?	
	☐ External audience ?
	☐ Enable Product Tag Inheritance ?
	☐ Internet accessible ?
	☐ Enable simple risk acceptance ?
	☒ Enable full risk acceptance ?

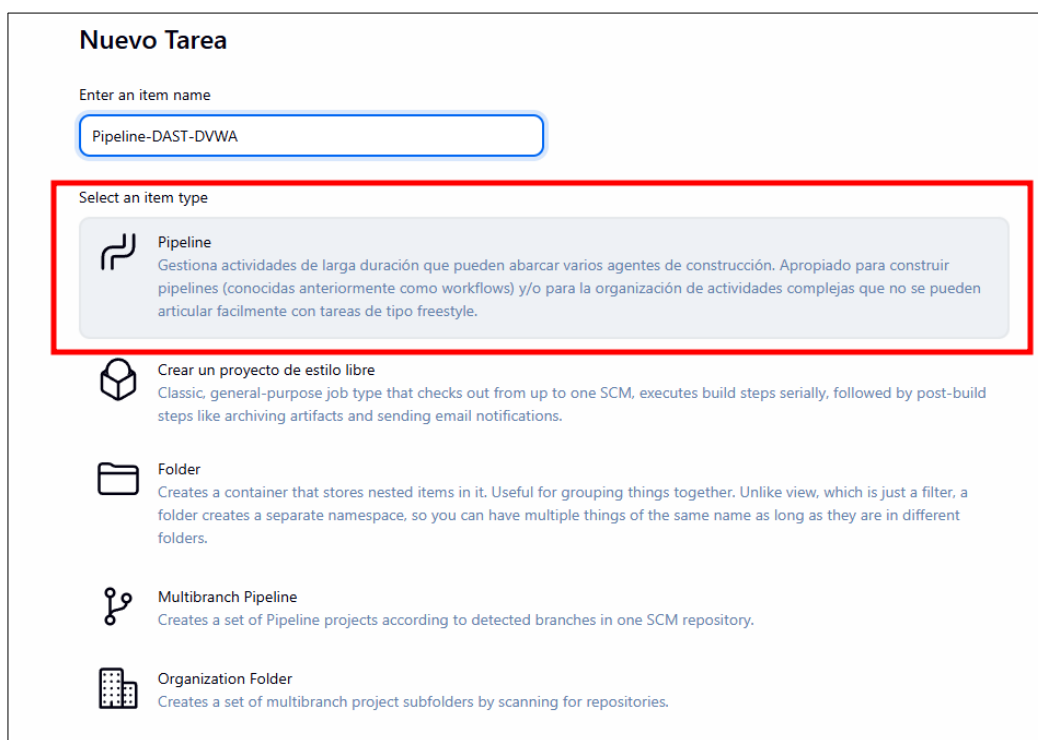
4. Añadir el Engagement en DefectDojo

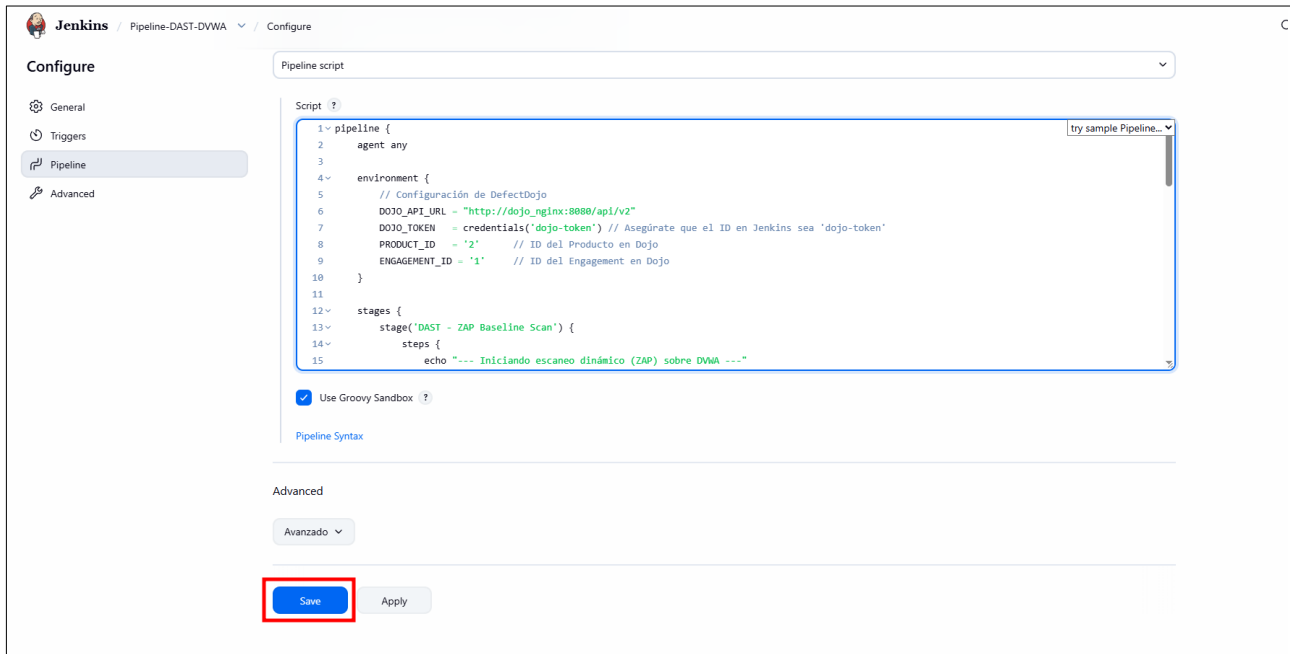
Dentro del producto DVWA se crea un Engagement de tipo CI/CD que representará el pipeline de análisis automatizado. Para ello se accede a Engagements > Add New CI/CD Engagement y se rellena con el nombre Pipeline-DAST-DVWA y la descripción Análisis dinámico DVWA con integración desde Jenkins.



5. Creación del Pipeline en Jenkins

Con la configuración de DefectDojo completada, se procede a crear el pipeline en Jenkins. Se selecciona Nueva Tarea, se asigna el nombre Pipeline-DAST-DVWA y se elige el tipo Pipeline. En la sección Script se introduce el código del pipeline, que consta de dos etapas: la primera lanza OWASP ZAP en modo baseline scan contra DVWA generando un informe XML, y la segunda sube automáticamente dicho informe a DefectDojo mediante su API REST v2. Una vez introducido el script se guarda y se ejecuta pulsando Construir ahora.





Jenkins / Pipeline-DAST-DVWA / Configure

Configure

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   environment {
5     // Configuración de DefectDojo
6     DOJO_API_URL = "http://dojo.nginx:8080/api/v2"
7     DOJO_TOKEN   = credentials('dojo-token') // Asegúrate que el ID en Jenkins sea 'dojo-token'
8     PRODUCT_ID   = '2' // ID del Producto en Dojo
9     ENGAGEMENT_ID = '1' // ID del Engagement en Dojo
10  }
11
12  stages {
13    stage('DAST - ZAP Baseline Scan') {
14      steps {
15        echo "---- Iniciando escaneo dinámico (ZAP) sobre DVWA ----"

☒ Use Groovy Sandbox ?



Pipeline Syntax



Advanced

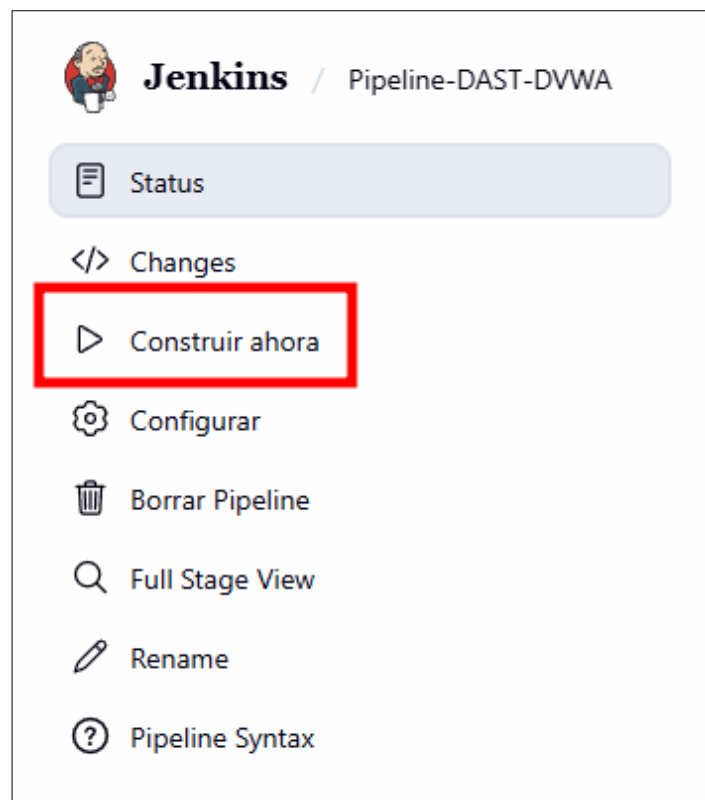


Avanzado



Save Apply


```



Jenkins / Pipeline-DAST-DVWA

Status

</> Changes

▶ Construir ahora

⚙️ Configurar

🗑️ Borrar Pipeline

🔍 Full Stage View

✎ Rename

❓ Pipeline Syntax

6. Verificación de resultados en DefectDojo

Tras la ejecución del pipeline, se accede a DefectDojo y se navega hasta el producto DVWA para verificar que los resultados han sido importados correctamente. En la sección Findings se pueden observar los hallazgos detectados por ZAP, clasificados por severidad, junto con los endpoints vulnerables identificados durante el escaneo.

DVWA

Overview

Components

Metrics

Engagements 1

Findings 21

Hosts / Endpoints 1 / 12

Benchmarks

Settings

Engagements / Pipeline-DAST-DVWA / View CI/CD Engagement

Description

Análisis dinámico DVWA con integración desde Jenkins

Tests (1) Critical: 0, High: 0, Medium: 5, Low: 9, Info: 7, Total: 21 Active Findings

Showing entries 1 to 1 of 1

Title / Type	Date	Lead	Total Findings	Active (Verified)	Mitigated	Duplicates	Notes	Reimports
ZAP Scan	May 6, 2026 - May 6, 2026		21	21 (21)	0	0		0

Showing entries 1 to 1 of 1

Risk Acceptance

No Risk Acceptances found.

Additional Features

Pipeline-DAST-DVWA

StatusIn Progress

Dates6th May - 13th May

Length7 days

Service Account(pedro)

TrackerNot Specified

RepoNot Specified

Test StrategyNot Specified

Updateda minute ago

Created2 minutes ago

CI/CD Engagement Details

Build IDNot Specified

Commit HashNot Specified

Branch/TagNot Specified

DVWA

Overview

Components

Metrics

Engagements 1

Findings 21

Hosts / Endpoints 1 / 12

Benchmarks

Settings

Open Findings

Showing entries 1 to 21 of 21

Column visibility

Copy

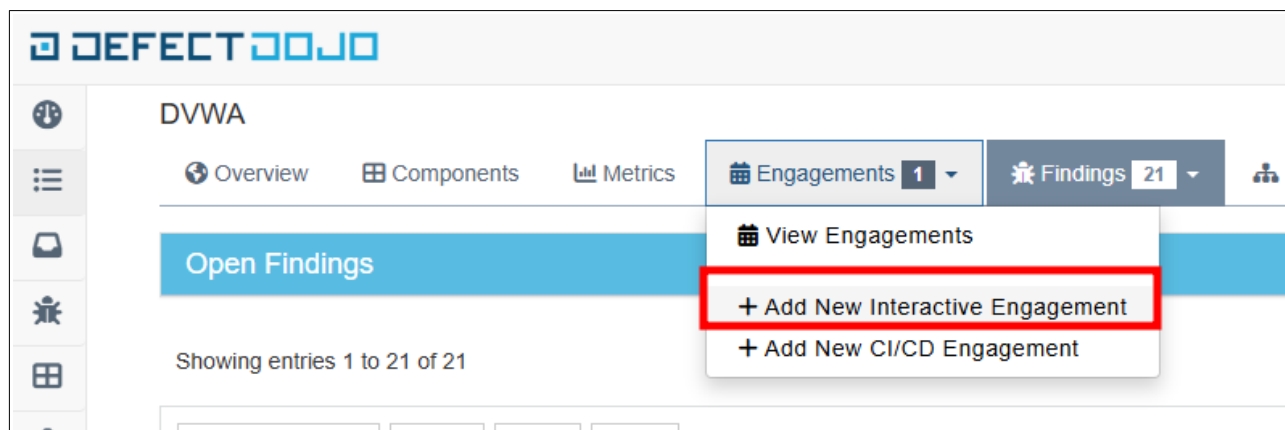
PDF

Print

	Severity	Name	CWE	Vulnerability Id	EPSS Score	EPSS Percentile	Date	Age	SLA	Reporter	Found By	Status
☐	Medium	Directory Browsing	548		N.A.	N.A.	May 6, 2026	0	90	(pedro)	ZAP Scan	Active, Verified
☐	Medium	HTTP Only Site	311		N.A.	N.A.	May 6, 2026	0	90	(pedro)	ZAP Scan	Active, Verified
☐	Medium	Content Security Policy (CSP) Header Not Set	693		N.A.	N.A.	May 6, 2026	0	90	(pedro)	ZAP Scan	Active, Verified
☐	Medium	Missing Anti-Clickjacking Header	1021		N.A.	N.A.	May 6, 2026	0	90	(pedro)	ZAP Scan	Active, Verified
☐	Medium	Relative Path Confusion	20		N.A.	N.A.	May 6, 2026	0	90	(pedro)	ZAP Scan	Active, Verified

7. Añadir un Engagement Interactive para escaneos manuales

Una vez completado el pipeline automatizado con ZAP, se procede a añadir un segundo Engagement de tipo Interactive en el producto DVWA para registrar escaneos realizados de forma manual. Para ello se accede a Engagements > Add New Interactive Engagement y se rellena con el nombre DAST Antiguos y la descripción Test realizados en clases anteriores.



New Interactive Engagement

Name ?

Description ?

B I H | **“ ”** | **≡** | **≡** | **🔗** | **🖼️** | **📅** | **—** | **</>**

Test realizados en clases anteriores

Status*

Repo ?

☐ Deduplication within this engagement only ?

Tags ?

Product*

Import Scan Results **Add Tests** **Done**

8. Añadir un Test con Nikto

Dentro del nuevo Engagement se añade un Test para importar los resultados de un escaneo Nikto realizado previamente. Se accede al menú del Engagement y se selecciona Import Scan Results. Se rellena el formulario con los siguientes datos: Scan Completion Date, Minimum Severity Info, Scan Type Nikto Scan y Environment Production. Finalmente se selecciona el archivo con los resultados del escaneo y se pulsa Import Scan Results.

The screenshot shows the DVWA interface. At the top, there's a navigation bar with tabs: Overview, Components, Metrics, Engagements (selected), Findings, Hosts / Endpoints, Benchmarks, and Settings. Below the navigation bar, the breadcrumb trail is 'Engagements / DAST Antiguos / View Engagement'. The main content area has several sections: 'Description' (Test realizados en clases anteriores), 'Tests (0)' (Critical: 0, High: 0, Medium: 0, Low: 0, Info: 0, Total: 0 Active Findings), 'Risk Acceptance' (No Risk Acceptances found), and 'Additional Features'. A dropdown menu is open over the 'Tests' section, showing options: '+ Add Tests', 'Import Scan Results' (highlighted with a red box), 'View Open Findings', and 'View All Findings'. Below this, the 'Add Tests' form is visible, with fields for: Scan Completion Date, Minimum severity* (set to Info), Active* (Not specified (default)), Verified* (Not specified (default)), Scan type* (set to Nikto Scan), Environment* (set to Production), and Systems / Endpoints (Nothing selected).

The screenshot shows a 'Choose report file' dialog box. It has a text input field labeled 'Seleccionar archivo' with the value 'nikto_scan_result.xml' (highlighted with a red box). Below the input field, there are four checkboxes with labels: 'Close old findings within this engagement', 'Close old findings within this product', 'Apply Tags to Findings', and 'Apply Tags to Endpoints'. All checkboxes are currently unchecked.

9. Verificación de resultados en DefectDojo

Tras importar los resultados de ambos escaneos, se accede al producto DVWA para verificar que todos los hallazgos han sido centralizados correctamente en DefectDojo. En la sección Findings se pueden observar todos los hallazgos detectados tanto por ZAP como por Nikto, clasificados por severidad, permitiendo tener una visión global del estado de seguridad de la aplicación.

The screenshot displays the DefectDojo web application interface. At the top, the 'DVWA' project is selected. The 'Engagements' tab is active, showing a table with one engagement: 'DAST Antiguos' in the 'Production' environment, dated 'May 6, 2026 - May 6, 2026', with a progress of 100% and 1 reimport. Below this, the 'Findings' section is expanded, showing 12 findings. The findings are listed in a table with columns: Severity, Name, CWE, Vulnerability Id, EPSS Score, EPSS Percentile, Date, Age, SLA, Reporter, Status, Group, and Planned Remediation. The findings are categorized by severity: Medium (3) and Info (9).

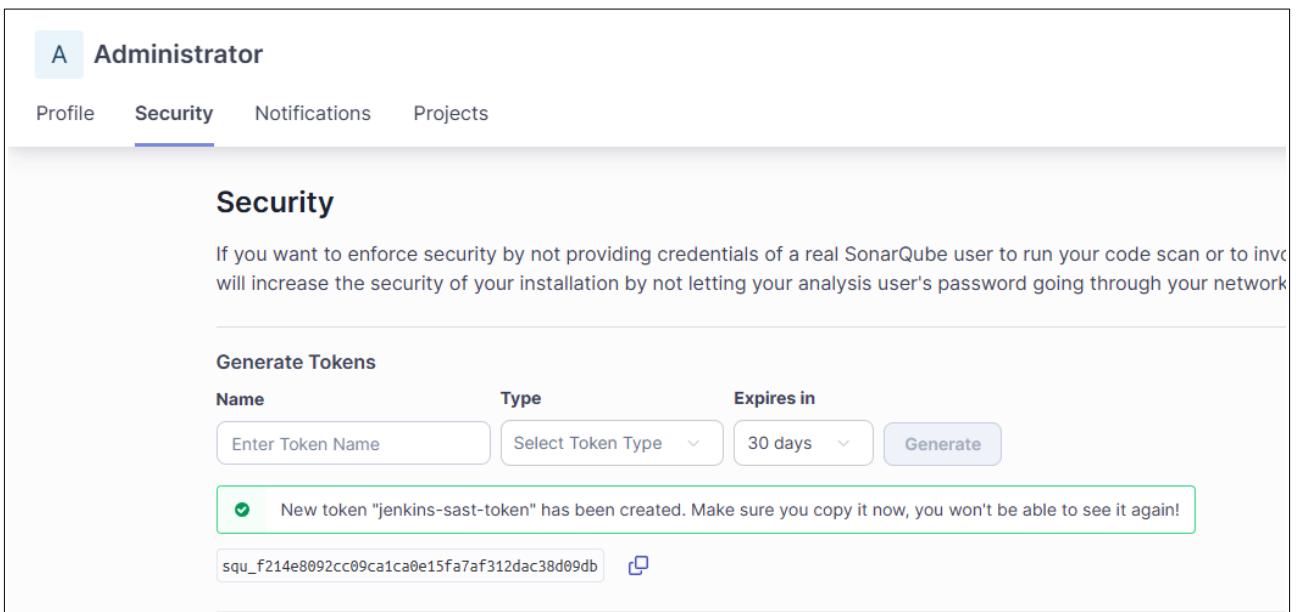
Severity	Name	CWE	Vulnerability Id	EPSS Score	EPSS Percentile	Date	Age	SLA	Reporter	Status	Group	Planned Remediation
Medium	/Docs/ Directory Indexing Found			N.A.	N.A.	May 6, 2026	0	90	(pedro)	Active		
Medium	/icons/Readme: Apache Default File Found			N.A.	N.A.	May 6, 2026	0	90	(pedro)	Active		
Medium	/Config/ Directory Indexing Found			N.A.	N.A.	May 6, 2026	0	90	(pedro)	Active		
Info	Cookie Security Created Without the HttpOnly Flag			N.A.	N.A.	May 6, 2026	0		(pedro)	Active		
Info	/login.php: Admin Login Page/Section Found			N.A.	N.A.	May 6, 2026	0		(pedro)	Active		

Actividad 3

SAST con SonarQube

1. Generación del token en SonarQube

El primer paso consiste en generar un token de autenticación en SonarQube para permitir la comunicación con Jenkins. Para ello se accede a SonarQube con el usuario administrador y se navega hasta My Account > Security > Generate Token. Se rellena el formulario con el nombre jenkins-sast-token, tipo User Token y sin fecha de expiración. Se pulsa Generate y se copia el token inmediatamente, ya que SonarQube no volverá a mostrarlo.



A Administrator

Profile **Security** Notifications Projects

Security

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke analysis, you will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name	Type	Expires in	
Enter Token Name	Select Token Type	30 days	Generate

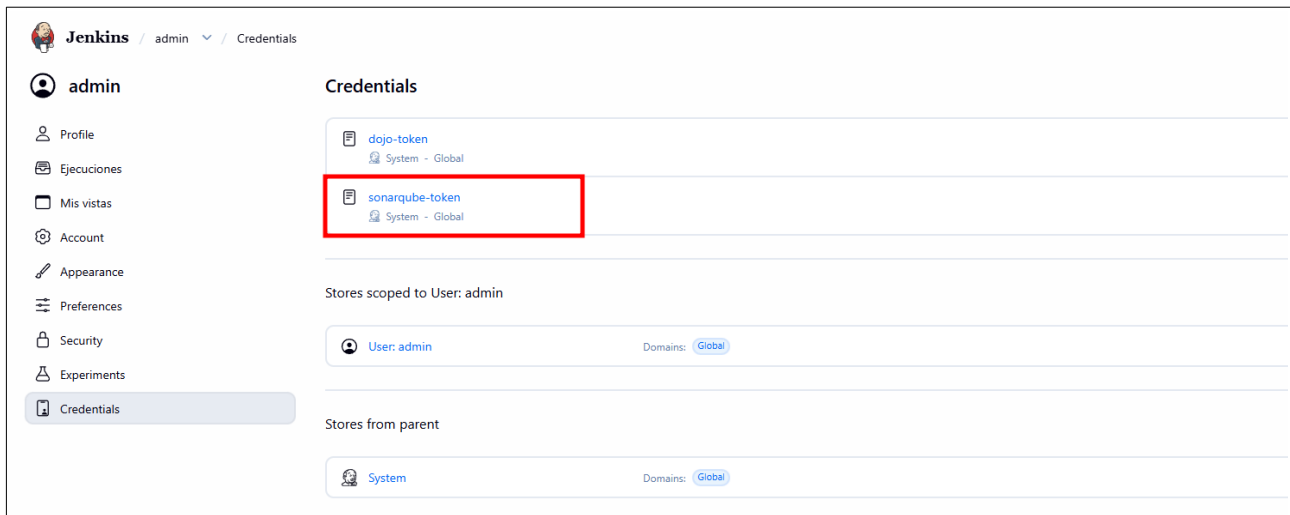
 New token "jenkins-sast-token" has been created. Make sure you copy it now, you won't be able to see it again!



Token: squ_f214e8092cc09ca1ca0e15fa7af312dac38d09db

2. Configuración del token de SonarQube en Jenkins

Con el token copiado, se accede a Jenkins y se navega hasta Administrar Jenkins > Credentials > Sistema > Global > Add Credentials. Se selecciona el tipo Secret Text, se pega el token en el campo Secret y se asigna el identificador sonarqube-token en el campo ID. Se pulsa Create para guardar la credencial.



3. Configuración del servidor SonarQube en Jenkins

A continuación se configura la conexión de Jenkins con el servidor SonarQube. Se accede a Administrar Jenkins > System y en la sección SonarQube Servers se añade un nuevo servidor con el nombre SonarQube y la URL de la instancia. Se selecciona el token sonarqube-token creado en el paso anterior como Server authentication token y se guarda la configuración.

(Esto ya lo hicimos en la preparación del entorno)

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

Instalaciones de SonarQube

[Listado de instalaciones SonarQube](#)

Name

SonarQube

URL del servidor

Por defecto es http://localhost:9000

http://192.168.100.116:9000

Server authentication token

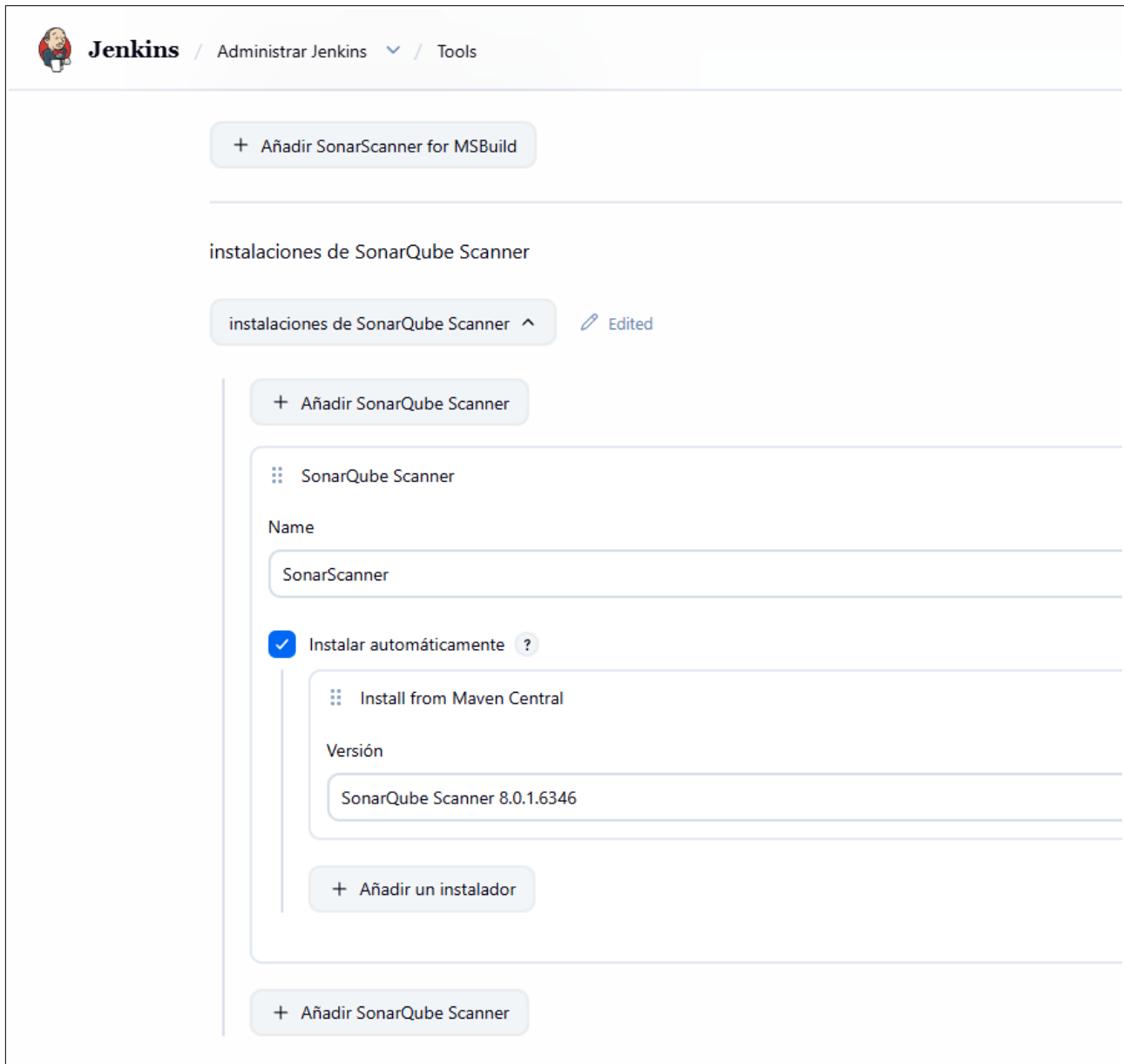
SonarQube authentication token. Mandatory when anonymous access is disabled.

sonarqube-token

Avanzado ▾

4. Configuración de SonarScanner en Jenkins

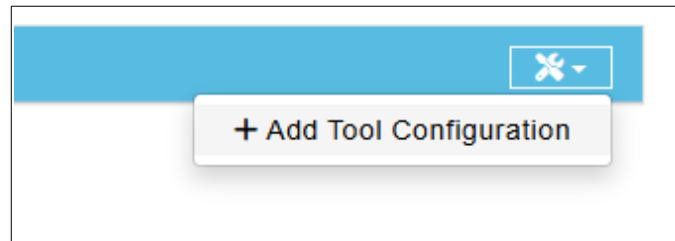
Se accede a Administrar Jenkins > Tools y se verifica que existe una instalación de SonarQube Scanner con el nombre SonarScanner. En caso de no estar instalado, se añade uno nuevo con instalación automática desde Maven Central con la versión más reciente disponible. Se guarda la configuración.



The screenshot shows the Jenkins 'Tools' configuration page for SonarQube Scanner. The breadcrumb navigation at the top reads 'Jenkins / Administrar Jenkins / Tools'. A button '+ Añadir SonarScanner for MSBuild' is visible at the top. Below, the section 'instalaciones de SonarQube Scanner' is shown with a dropdown menu and an 'Edited' status. A '+ Añadir SonarQube Scanner' button is present. The configuration for a 'SonarQube Scanner' tool is displayed, including a 'Name' field with the value 'SonarScanner'. The 'Instalar automáticamente' checkbox is checked, and the 'Install from Maven Central' option is selected. The 'Versión' field contains 'SonarQube Scanner 8.0.1.6346'. At the bottom, there is a '+ Añadir un instalador' button and another '+ Añadir SonarQube Scanner' button.

5. Configuración del servidor SonarQube en DefectDojo

Para centralizar los hallazgos de SonarQube en DefectDojo, se accede al menú de configuración de DefectDojo y se selecciona Tool Configurations > Add Tool Configuration.



Se rellena el formulario con el nombre SonarQube, Tool Type SonarQube, Authentication Type API Key y como URL se utiliza la IP interna del contenedor, que se obtiene ejecutando el siguiente comando:

```
$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' sonarqube
```

Se pega el token de SonarQube en el campo API Key y se guarda.

```
ciber@ciber:~$ docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' sonarqube
172.18.0.4
ciber@ciber:~$ |
```

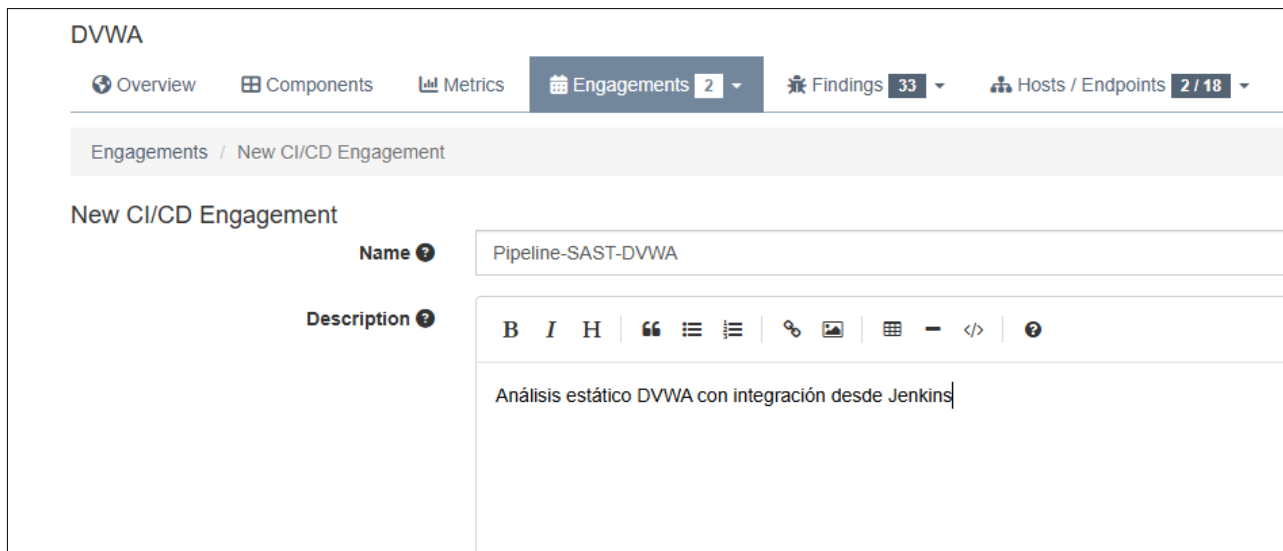
IP: 172.18.0.4

Add a Tool Configuration

Name*	SonarQube
Description	
Url	http://172.18.0.4:9000/api
Tool Type*	SonarQube
Authentication type	API Key
Extras ?	
Username	
Password	
Title for SSH/API Key	
SSH Key	
API Key	squ_f214e8092cc09ca1ca0e15fa7af312dac38d09db
Submit	

6. Creación del Engagement en DefectDojo

Sobre el mismo producto DVWA se crea un nuevo Engagement de tipo CI/CD para centralizar los hallazgos del análisis estático. Se accede a Engagements > Add New CI/CD Engagement y se rellena con el nombre Pipeline-SAST-DVWA y la descripción Análisis estático DVWA con integración desde Jenkins.



The screenshot shows the DefectDojo interface for creating a new engagement. At the top, the 'DVWA' product is selected. The navigation bar includes 'Overview', 'Components', 'Metrics', 'Engagements' (with a count of 2), 'Findings' (with a count of 33), and 'Hosts / Endpoints' (with a count of 2 / 18). The breadcrumb trail is 'Engagements / New CI/CD Engagement'. The form title is 'New CI/CD Engagement'. It contains two main input fields: 'Name' with the value 'Pipeline-SAST-DVWA' and 'Description' with the value 'Análisis estático DVWA con integración desde Jenkins'. The description field has a rich text editor toolbar with options for bold, italic, header, quote, list, link, image, table, and code.

DVWA

Overview Components Metrics Engagements 2 Findings 33 Hosts / Endpoints 2 / 18

Engagements / New CI/CD Engagement

New CI/CD Engagement

Name Pipeline-SAST-DVWA

Description

B I H | | | | | | |

Análisis estático DVWA con integración desde Jenkins

7. Creación del proyecto en SonarQube

Antes de ejecutar el pipeline, se crea manualmente el proyecto en SonarQube para que el Project Key coincida con el que se utilizará en el pipeline. Se accede a SonarQube > Projects > Create a local project y se rellena con el nombre dvwa-sast, Project Key dvwa-sast y rama principal main. En la siguiente pantalla se selecciona Use the global setting y se finaliza la creación.

1 of 2

Create a local project

Project display name *

 
Up to 255 characters. Some scanners might override the value you provide.

Project key *

 
The project key is a unique identifier for your project. It may contain up to 400 characters. Allowed characters are alphanumeric, '-' (dash), '_' (underscore), '.' (period) and ':' (colon), with at least one non-digit.

Main branch name *

The name of your project's default branch [Learn More](#) 

2 of 2

Set up project for Clean as You Code

The new code definition sets which part of your code will be considered new code. This You Code methodology. Learn more: [Defining New Code](#) 

Choose the baseline for new code for this project

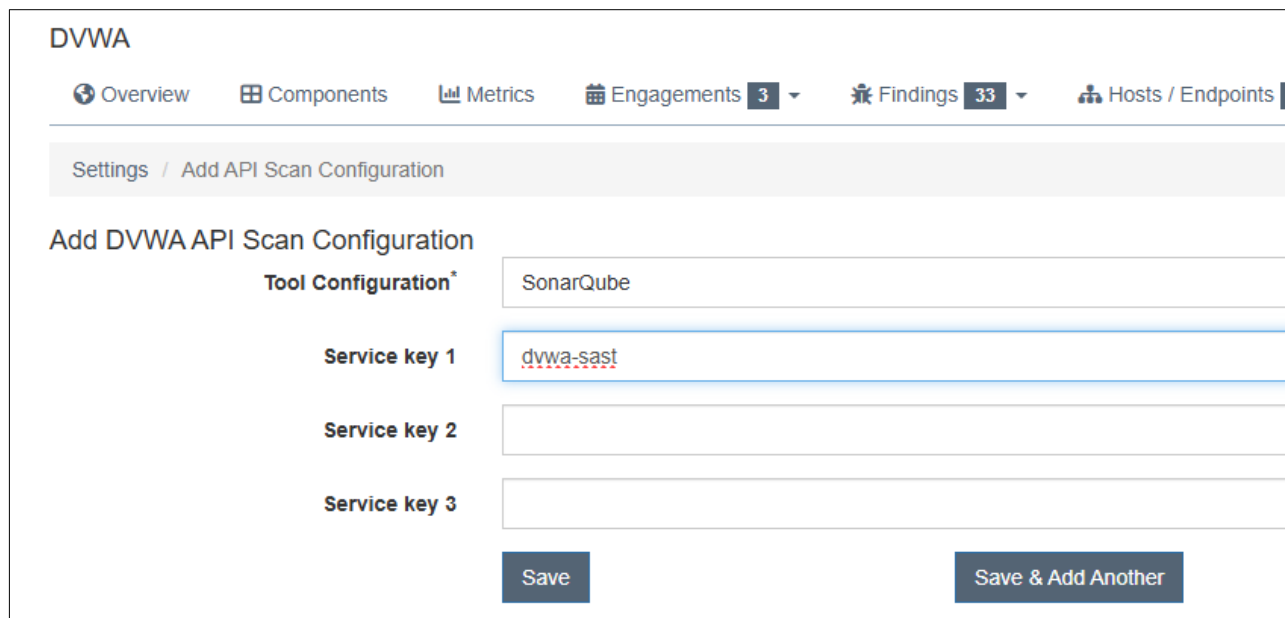
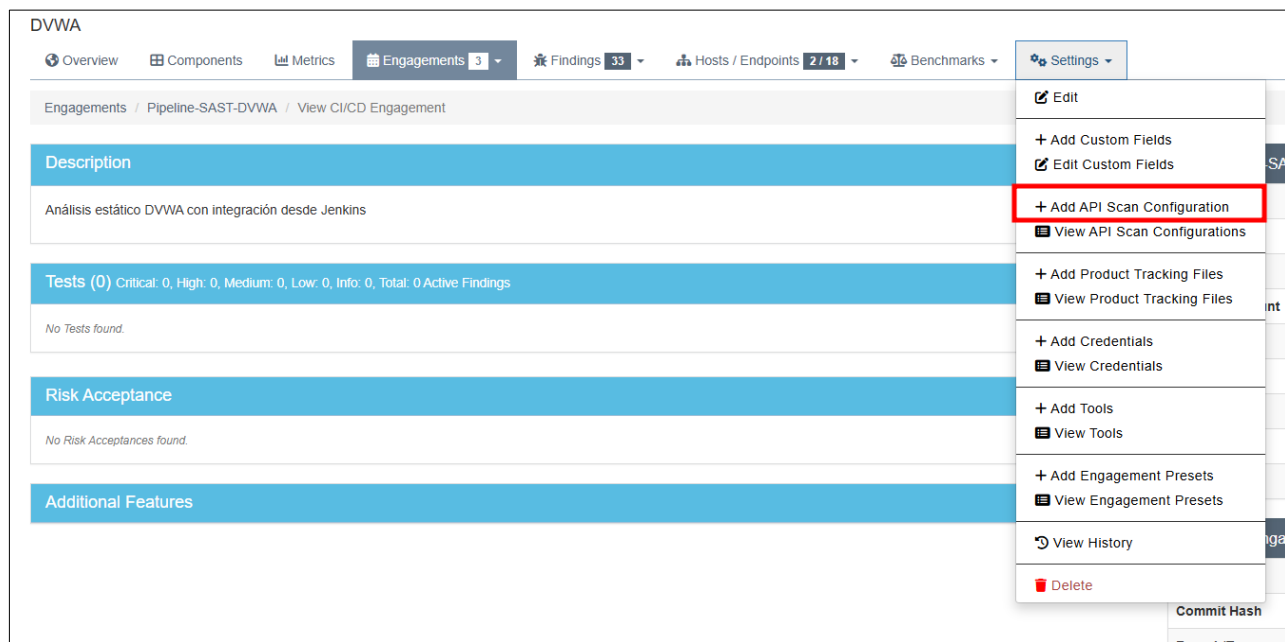
☒ **Use the global setting**

Previous version

Any code that has changed since the previous version is considered new code.
Recommended for projects following regular versions or releases.

8. Configuración de la API Scan Configuration en el Engagement

Dentro del Engagement Pipeline-SAST-DVWA se accede a Settings > Add API Scan Configuration. Se selecciona SonarQube como Tool Configuration y se escribe dvwa-sast en el campo Service key 1, que debe coincidir exactamente con el Project Key definido en SonarQube y en el pipeline.



9. Creación del Pipeline en Jenkins

Se crea una nueva tarea en Jenkins de tipo Pipeline con el nombre Pipeline-SAST-DVWA. En la sección Script se introduce el pipeline, que consta de las siguientes etapas: limpieza del workspace y clonado del repositorio de DVWA desde GitHub, análisis estático con SonarQube Scanner y envío de resultados a DefectDojo mediante su API REST. Se guarda y se ejecuta pulsando Construir ahora.

Nuevo Tarea

Enter an item name

Pipeline-SAST-DVWA

Select an item type



Pipeline

Gestiona actividades de larga duración que pueden abarcar varios agentes de construcción. Apropiado para construir pipelines (conocidas anteriormente como workflows) y/o para la organización de actividades complejas que no se pueden articular fácilmente con tareas de tipo freestyle.

Script ?

```
1 pipeline {
2   agent any
3
4   environment {
5     DOJO_API_URL    = "http://dojo_nginx:8080/api/v2"
6     PRODUCT_ID     = '1'
7     ENGAGEMENT_ID  = '3'
8   }
9
10  stages {
11    stage('Limpieza y Descarga') {
12      steps {
13        deleteDir()
14        sh "git clone https://github.com/digininja/DVWA.git ."
15      }
16    }
17  }
18 }
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

10. Verificación de resultados

Tras la ejecución del pipeline, se accede a DefectDojo y se navega hasta el producto DVWA para verificar que los hallazgos han sido importados correctamente desde SonarQube. En la sección Findings se pueden observar los hallazgos clasificados por severidad, incluyendo vulnerabilidades críticas, altas y medias detectadas en el código fuente de DVWA.

Active Engagements (2)

Showing entries 1 to 2 of 2

Column visibility

Copy

Excel

CSV

PDF

Print

Name	Type	Lead	Date	Length	Tests	Active (Verified)
Pipeline-SAST-DVWA	CI/CD	(pedro)	6th May - 13th May	7 days	1	74 (74)
Pipeline-DAST-DVWA	CI/CD	(pedro)	6th May - 13th May	7 days	1	21 (21)

Showing entries 1 to 2 of 2

Engagements / Pipeline-SAST-DVWA														
Open Findings														
Showing entries 1 to 25 of 74														
Column visibility Copy PDF Print														
Search:														
	Severity	Name	CWE	Vulnerability Id	EPSS Score	EPSS Percentile	Date	Age	SLA	Reporter	Found By	Status	Group	Service
☐	Critical	Make Sure This MySQL Database Password Gets Changed and Rem...	798		N.A.	N.A.	May 6, 2026	0	7	(pedro)	SonarQube API Import	Active, Verified		SonarQube
☐	Critical	Revoke and Change This Password, as It Is Compromised.	798		N.A.	N.A.	May 6, 2026	0	7	(pedro)	SonarQube API Import	Active, Verified		SonarQube
☐	High	Make Sure No Write Permissions Are Assigned to the Executab...	732		N.A.	N.A.	May 6, 2026	0	30	(pedro)	SonarQube API Import	Active, Verified		SonarQube
☐	High	Make Sure That Formatting This SQL Query Is Safe Here.	20		N.A.	N.A.	May 6, 2026	0	30	(pedro)	SonarQube API Import	Active, Verified		SonarQube

Actividad 4

SAST con Semgrep

1. Consideraciones previas

A diferencia de las prácticas anteriores, en esta práctica no es necesario realizar ninguna configuración adicional en Jenkins ni en DefectDojo, ya que el token de autenticación de DefectDojo (dojo-token) ya se encuentra registrado como credencial global en Jenkins desde la Actividad 2. Por tanto, únicamente se procede a la creación del pipeline.

2. Creación del Engagement en DefectDojo

Antes de crear el pipeline, se crea un nuevo Engagement de tipo CI/CD en el producto DVWA para centralizar los hallazgos del análisis con Semgrep. Se accede a Engagements > Add New CI/CD Engagement y se rellena con el nombre Pipeline-SAST-DVWA-Semgrep y la descripción Análisis estático DVWA con Semgrep desde Jenkins. Una vez creado, se anota el ID del engagement que aparece en la URL del navegador, ya que se utilizará en el pipeline.



New CI/CD Engagement	
Name ?	Pipeline-SAST-DVWA-Semgrep
Description ?	B I H Análisis estático DVWA con Semgrep desde Jenkins

<http://192.168.100.116:8082/engagement/4>

3. Creación del Pipeline en Jenkins

Se accede a Jenkins y se selecciona Nueva Tarea. Se asigna el nombre Pipeline-SAST-DVWA-Semgrep y se elige el tipo Pipeline. En la sección Script se introduce el pipeline, que define las variables de entorno necesarias: la URL de la API de DefectDojo (http://dojo_nginx:8080/api/v2), el token de autenticación mediante la credencial dojo-token, el PRODUCT_ID con valor 1 y el ENGAGEMENT_ID con el ID obtenido del engagement creado en el paso anterior. El pipeline consta de tres etapas: limpieza del workspace y clonado del repositorio de DVWA desde GitHub, ejecución del análisis estático con Semgrep generando un informe en formato JSON, y subida automática de dicho informe a DefectDojo mediante su API REST. Se guarda la configuración y se ejecuta pulsando Construir ahora.

Nueva Tarea

Enter an item name

Pipeline-SAST-DVWA-Semgrep

Select an item type

 **Pipeline**
Gestiona actividades de larga duración que pueden abarcar varios agentes de construcción. Apropiado para construir pipelines (conocidas anteriormente como workflows) y/o para la organización de actividades complejas que no se pueden articular fácilmente con tareas de tipo freestyle.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   // Definición de variables globales para todo el pipeline
5   environment {
6     // URL interna de Docker: Jenkins habla con el contenedor 'dojo_nginx' por el puerto 8080
7     DOJO_API_URL = "http://dojo_nginx:8080/api/v2"
8     // Credencial guardada en Jenkins (Secret Text) con el Token de la API de Dojo
9     DOJO_TOKEN = credentials('dojo-token')
10    // IDs de la base de datos de DefectDojo para organizar el reporte
11    PRODUCT_ID = '1'
12    ENGAGEMENT_ID = '4'
13  }
14
15  stages {
```

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

4. Verificación de resultados en DefectDojo

Tras la ejecución del pipeline, se accede a DefectDojo y se navega hasta el producto DVWA, Engagement Pipeline-SAST-DVWA. En la sección de tests se puede observar que se ha creado un nuevo test de tipo Semgrep JSON Report. Los resultados muestran un total de 70 findings, distribuidos en 44 de severidad alta y 26 de severidad media, destacando vulnerabilidades relacionadas con inyección SQL, uso de funciones peligrosas como exec y eval, y uso de algoritmos de hashing débiles como MD5.

The screenshot displays the DefectDojo interface for the DVWA engagement. The top navigation bar includes links for Overview, Components, Metrics, Engagements (4), Findings (177), Hosts / Endpoints (2 / 18), Benchmarks, and Settings. The breadcrumb trail indicates the path: Engagements / Pipeline-SAST-DVWA-Semgrep / View CI/CD Engagement.

Description

Análisis estático DVWA con Semgrep desde Jenkins

Tests (1) Critical: 0, High: 44, Medium: 26, Low: 0, Info: 0, Total: 70 Active Findings

Showing entries 1 to 1 of 1

Title / Type	Date	Lead	Total Findings	Active (Verified)	Mitigated	Duplicates	Notes	Reimports
Semgrep JSON Report	May 6, 2026 - May 6, 2026		70	70 (70)	0	0		0

Showing entries 1 to 1 of 1

Risk Acceptance

No Risk Acceptances found.

Additional Features

Actividad 5

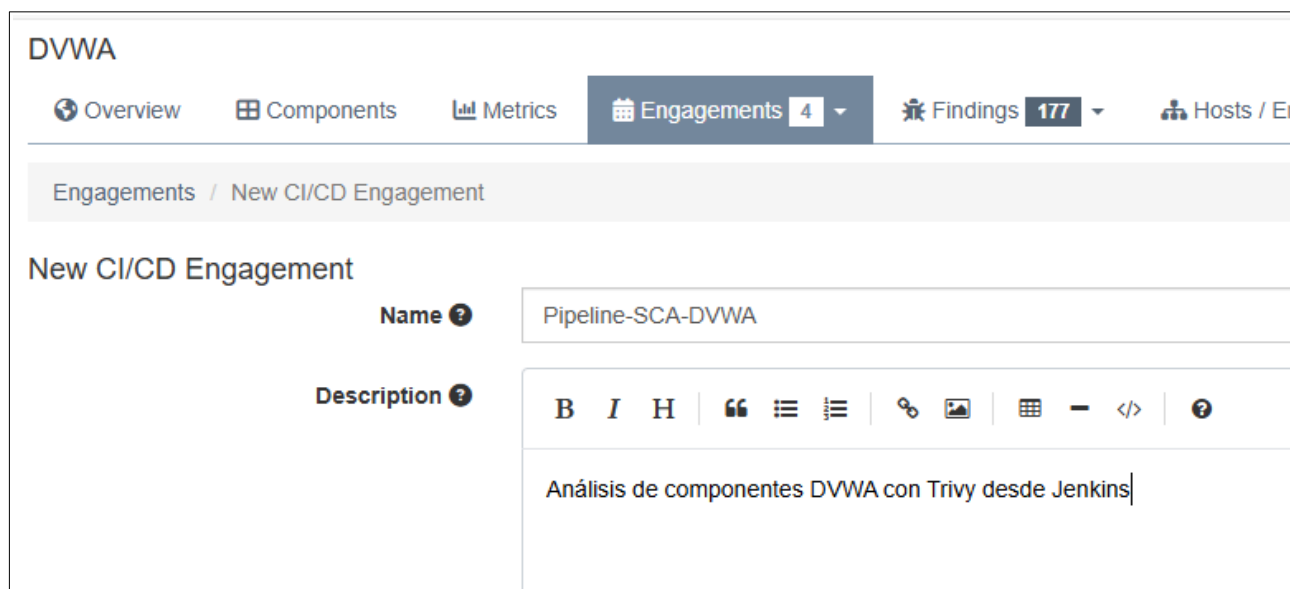
SCA con Trivy

1. Consideraciones previas

Al igual que en la práctica anterior, no es necesario realizar ninguna configuración adicional en Jenkins ya que el token de DefectDojo está disponible como credencial global. Únicamente se crea el Engagement en DefectDojo y el pipeline en Jenkins.

2. Creación del Engagement en DefectDojo

Se accede al producto DVWA en DefectDojo y se crea un nuevo Engagement de tipo CI/CD para centralizar los hallazgos del análisis de componentes. Se accede a Engagements > Add New CI/CD Engagement y se rellena con el nombre Pipeline-SCA-DVWA y la descripción Análisis de componentes DVWA con Trivy desde Jenkins. Una vez creado, se anota el ID del engagement que aparece en la URL del navegador, en este caso el valor 5.



The screenshot shows the DefectDojo DVWA interface. At the top, there's a navigation bar with tabs: Overview, Components, Metrics, Engagements (4), Findings (177), and Hosts / E. Below this, a breadcrumb trail reads 'Engagements / New CI/CD Engagement'. The main section is titled 'New CI/CD Engagement'. It contains two input fields: 'Name' with the value 'Pipeline-SCA-DVWA' and 'Description' with the value 'Análisis de componentes DVWA con Trivy desde Jenkins'. The description field has a rich text editor toolbar above it with icons for bold, italic, header, quote, list, link, image, table, and code.

3. Creación del Pipeline en Jenkins

Se accede a Jenkins y se selecciona Nueva Tarea. Se asigna el nombre Pipeline-SCA-DVWA y se elige el tipo Pipeline. En la sección Script se introduce el pipeline, que define las variables de entorno: la URL de la API de DefectDojo (http://dojo_nginx:8080/api/v2), el token de autenticación mediante la credencial dojo-token, el PRODUCT_ID con valor 1 y el ENGAGEMENT_ID con valor 4. El pipeline ejecuta Trivy contra la imagen Docker de DVWA generando un informe en formato JSON y lo sube automáticamente a DefectDojo mediante su API REST. Se guarda la configuración y se ejecuta pulsando Construir ahora.

Nuevo Tarea

Enter an item name

Pipeline-SCA-DVWA

Select an item type

**Pipeline**

Gestiona actividades de larga duración que pueden abarcar varios agentes de construcción. Apropiado para construir pipelines (conocidas anteriormente como workflows) y/o para la organización de actividades complejas que no se pueden articular fácilmente con tareas de tipo freestyle.

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

Script ?

```
1 pipeline {
2   agent any
3
4   // Definición de variables globales para todo el pipeline
5   environment {
6     // URL interna de Docker: Jenkins habla con el contenedor 'dojo_nginx' por el puerto 8080
7     DOJO_API_URL = "http://dojo_nginx:8080/api/v2"
8     // Credencial guardada en Jenkins (Secret Text) con el Token de la API de Dojo
9     DOJO_TOKEN = credentials('dojo-token')
10    // IDs de la base de datos de DefectDojo para organizar el reporte
11    PRODUCT_ID = '1'
12    ENGAGEMENT_ID = '5'
13  }
14
15  stages {
```

☒ Use Groovy Sandbox ?

4. Verificación de resultados en DefectDojo

Tras la ejecución del pipeline, se accede a DefectDojo y se navega hasta el producto DVWA, Engagement Pipeline-SCA-DVWA. En la sección de tests se puede observar que se ha creado un test de tipo Trivy Scan. Los resultados muestran un total de 1576 findings distribuidos en 254 críticos, 552 altos, 642 medios, 116 bajos y 12 informativos, correspondientes a vulnerabilidades CVE detectadas en los paquetes del sistema operativo y dependencias del contenedor de DVWA.

DVWA

Overview

Components

Metrics

Engagements 5

Findings 1753

Hosts / Endpoints 2 / 18

Benchmarks

Settings

Engagements / Pipeline-SCA-DVWA / Trivy Scan / Test

Trivy Scan

Updated 52 seconds ago, Created 3 minutes ago

Engagement	Environment	Dates	Updated	Progress	Version
Pipeline-SCA-DVWA	Development	May 6, 2026 - May 6, 2026	May 6, 2026	100%	

Import History (1)

Groups (0)

Findings (1576)

Critical: 254, High: 552, Medium: 642, Low: 116, Info: 12, Total: 1576 Findings

Showing entries 1 to 25 of 1576

Column visibility

Copy

Excel

CSV

PDF

Print

	Severity	Name	CWE	Vulnerability Id	EPSS Score	EPSS Percentile	Date	Age	SLA	Reporter	Status
☐	Critical	CVE-2021-39275 Apache2-Data 2.4.25-3+deb9u5 	 787	 CVE-2021-39275	N.A.	N.A.	May 6, 2026	0	 7	(pedro)	Active, Verified
☐	Critical	CVE-2022-23943 Apache2-Data 2.4.25-3+deb9u5 	 190	 CVE-2022-23943	N.A.	N.A.	May 6, 2026	0	 7	(pedro)	Active, Verified